

08 — Master Coding-Agent Handoff & Final Decisions

The final bridge between the CampusFlow source documents and Antigravity / coding agents

CampusFlow • Final Implementation Decision • October 2026

1. Purpose of this document

This document is the final handoff and decision layer for the existing CampusFlow source documents. The seven earlier documents define the product, UX, frontend, API, data model, security, AI/OCR pipeline and setup. This document closes the remaining implementation decisions so a coding agent can start without repeatedly asking the team to choose technologies or product behavior.

Decision rule: when this document conflicts with an older optional suggestion, this document is the final implementation decision for the hackathon MVP. The original official college documents remain the source of truth for academic facts.

2. Final Product Definition

Product: CampusFlow

Tagline: Everything important on campus. One organized place.

Platform: Responsive web application.

Primary users: Students.

Secondary users: Faculty / coordinators and administrators.

Core promise: A student can quickly understand what changed, what they need to do, when it is due, and what opportunities are available.

3. Final MVP Scope — Build This

- Student login using college email + password.
- Student dashboard with personalized priority information.
- Official notices with search, filters, priority and unread state.
- Notice detail with original source document/link.
- Deadlines with upcoming/overdue state.
- Events with eligibility, capacity and registration deadline.
- Transaction-safe event registration and duplicate prevention.
- Calendar combining events and deadlines.
- In-app notifications.
- Student profile.
- Admin dashboard.
- Admin notice create/edit/publish/archive.
- Audience targeting: Department → Year → Division → Batch.
- Admin event create/edit/publish.
- Registration management.
- Audit logs.
- PDF/image upload for official documents.
- PDF text extraction and OCR for scanned documents.
- Structured extraction of what changed, audience, action and explicit dates.
- Mandatory human review before extracted official information is published.
- AI-generated notice summary with explicit AI disclosure.
- Basic controlled ingestion from the official college Notice Board / News Announcement source.

4. Explicitly NOT in MVP

- WhatsApp API integration.
- Push notifications.
- Complex recommendation engine.

- Semantic/vector search.
- Real-time chat.
- Student attendance management.
- Marks/result management.
- Fees/payment management.
- Full ERP replacement.
- Automated publication of AI/OCR output without human review.
- Redis unless a measured requirement appears.
- Microservice architecture.

These can be Phase 2+. The hackathon product should demonstrate one complete and reliable workflow instead of many unfinished systems.

5. Final Technology Decisions

Layer	Final decision	Reason
Frontend	React + TypeScript + Vite	Fast development, strong component model
Styling	Tailwind CSS + custom tokens	Consistent responsive UI
Icons	Lucide React	Accessible consistent icon set
Routing	React Router	Simple SPA routing
Server state	TanStack Query	Caching/loading/error/invalidation
Forms	React Hook Form + Zod	Validation and predictable forms
Backend	Node.js + TypeScript + Express	Familiar modular REST API
Database	PostgreSQL on Neon	Managed relational database
ORM	Prisma	Typed schema and migrations
Auth	Secure httpOnly session cookie	Final decision over JWT for MVP
Password	Argon2id	Strong password hashing
Validation	Zod	Shared conceptual validation
File storage	Cloudinary	Simple PDF/image storage for demo/early production
AI	Google Gemini API	Simple structured-output integration
OCR	Tesseract.js	Local/server OCR path without a second paid OCR service
Scraping	Cheerio; Playwright only if needed	Lightweight official-page ingestion
Frontend hosting	Vercel	Simple React deployment
Backend hosting	Render	Simple Node service deployment
Database hosting	Neon	Managed PostgreSQL
Monitoring	Sentry, optional but recommended	Error visibility
Cache/queue	None in MVP	Avoid unnecessary infrastructure
Testing	Vitest + Supertest + Playwright	Unit/API/E2E coverage
Containers	Docker for backend/deployment portability	Repeatable runtime

6. Final Authentication Decision

Authentication is **college email + password**. Do not use PRN as the login identifier.

Allowed email domain: @moderncoe.edu.in.

Normalize imported and login emails by removing all whitespace and converting to lowercase.

```
const normalized = email.replace(/\s/g, "").toLowerCase();
```

Use secure httpOnly cookies. Student session target: 8 hours. Admin session target: 2 hours. Rate-limit login attempts.

For the hackathon, use seeded demo accounts. Do not use real student passwords in the repository.

7. Final Academic Data Model

Use the hierarchy required by the supplied CampusFlow materials:

```
College
  → Department: MCA, MBA
    → Year: FY, SY
      → Division: A, B
        → Batch: F1, F2, F3
```

The supplied source materials describe F1/F2/F3 and FY/SY MCA structures. The implementation should keep the hierarchy configurable rather than hard-coding only one division or batch.

8. Final Demo Data Decision

Use fictional/anonymized development records for the coding agent and Git repository.

```
student_f1_01@moderncoe.edu.in
student_f1_02@moderncoe.edu.in
student_f2_01@moderncoe.edu.in
student_f3_01@moderncoe.edu.in
faculty_mca@moderncoe.edu.in
admin@moderncoe.edu.in
```

Development password: store only through environment/seed configuration; never hard-code production credentials. If a visible demo credential is needed, use a clearly temporary value and change it before any real deployment.

The real college email CSV supplied for the project must stay outside Git and outside public deployment. It is input data, not source code.

9. Final Official-Document Workflow

```
ADMIN UPLOAD / OFFICIAL WEBSITE
↓
FILE VALIDATION
↓
SHA-256 HASH + DUPLICATE CHECK
↓
PDF TEXT EXTRACTION
↓
OCR IF SCANNED
↓
RAW SOURCE TEXT STORED
↓
STRUCTURED EXTRACTION
↓
HUMAN REVIEW
↓
PUBLISH
↓
AI SUMMARY
↓
PERSONALIZED STUDENT VIEW
```

Original PDF/source is authoritative. AI and OCR are assistance layers. No extracted academic fact becomes official CampusFlow data until authorized review.

10. Final AI Contract

Input: official source text.

Output JSON:

```
{
  "what_changed": "string",
  "who_is_affected": "string",
  "required_action": "string|null",
  "deadline": "ISO date|null",
  "audience_hint": {
    "department": "string|null",
    "year": "FY|SY|null",
    "division": "A|B|null",
  }
}
```

```

    "batch": "F1|F2|F3|null"
  }
}

```

- Never invent a date, requirement, penalty, room, eligibility rule or deadline.
- If information is absent, return null.
- Validate the response with Zod.
- Evidence-check dates against the raw source text.
- Store original source and AI summary separately.
- Display AI-generated summary disclosure.
- If AI fails, the original notice still works.

11. Final Scraper Decision

Target the official college website only for relevant public notice/news sources. Planned first sources: Notice Board and News Announcement.

- Schedule: every 30 minutes in production-like deployment; allow manual Run Now for demo.
- Download only supported document types.
- Hash every downloaded file.
- Skip already-known hashes.
- Store source URL and retrieval timestamp.
- Do not aggressively crawl the website.
- Do not bypass authentication or access restrictions.
- New scraped content enters review; it does not auto-publish.

12. Final Roles & Permissions

Capability	Student	Faculty	Admin
Read eligible notices/events/deadlines	Yes	Yes	Yes
Register for events	Yes	Yes	Yes
Create notice	No	Yes	Yes
Publish notice	No	Yes	Yes
Create event	No	Yes	Yes
Manage registrations	No	Own/assigned	Yes
Manage users	No	No	Yes
Change roles	No	No	Yes
Audit logs	No	No	Yes
System settings	No	No	Yes

13. Final Repository Structure

```

campusflow/
├── frontend/
├── backend/
├── docs/
├── 01_product_design.pdf
├── 02_frontend_architecture.pdf
├── 03_api_contract.pdf
├── 04_data_model_and_logic.pdf
├── 05_security_auth.pdf
├── 06_ai_scraping_pipeline.pdf
├── 07_setup_and_api_keys.pdf
└── 08_master_handoff.pdf

```

```

■■■ source-material/
■   ■■■ academic-calendar.pdf
■   ■■■ timetables/
■   ■■■ sample-notices/
■   ■■■ README.md
■■■ .gitignore
■■■ README.md
■■■ docker-compose.yml

```

The real student CSV is not placed in source-material if it contains personal data. Keep it outside Git and import it locally when authorized.

14. Required Source Materials for the Agent

- CampusFlow_Product_Design_and_Technical_Specification.pdf — original source of truth.
- Seven production PDFs already generated.
- Academic Calendar FY MCA & MBA 2026–27 Term I.
- FY MCA timetable documents/images.
- Any sample official notice PDFs/images.
- Anonymized development student data.
- Official college website URL: <https://www.moderncoe.edu.in/>

The coding agent must read the product/technical PDFs before writing application code. The supplied official academic documents are test fixtures for document ingestion and must not be altered.

15. Environment Variables

```

DATABASE_URL=
AUTH_SECRET=
APP_URL=http://localhost:5173
API_URL=http://localhost:3000
AI_PROVIDER_KEY=
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=
SENTRY_DSN=

```

Do not commit .env. Commit only .env.example with empty values.

16. Development Sequence — Agent Must Follow

```

STEP 0 — READ ALL DOCS + SOURCE MATERIAL
↓
STEP 1 — SCAFFOLD REPOSITORY
↓
STEP 2 — DATABASE + MIGRATIONS + SEED
↓
STEP 3 — AUTH + RBAC
↓
STEP 4 — STUDENT DASHBOARD
↓
STEP 5 — NOTICES + DEADLINES
↓
STEP 6 — EVENTS + TRANSACTION-SAFE REGISTRATION
↓
STEP 7 — ADMIN PUBLISHING + AUDIENCE TARGETING
↓
STEP 8 — PDF UPLOAD + EXTRACTION + OCR + REVIEW
↓
STEP 9 — AI SUMMARY
↓
STEP 10 — CONTROLLED SCRAPER
↓
STEP 11 — NOTIFICATIONS + CALENDAR
↓
STEP 12 — TEST + SECURITY + RESPONSIVE QA
↓
STEP 13 — DEPLOY DEMO
↓
STEP 14 — FINAL DEMO REHEARSAL

```

After every major step: run the application, run tests, inspect the UI, fix errors, then continue. Do not build the entire product first and test at the end.

17. Hackathon Demo Script

- 1. Open CampusFlow landing/login.
- 2. Log in as Admin.
- 3. Upload the supplied academic calendar PDF.
- 4. Show extracted source text/structured fields.
- 5. Show human review screen.
- 6. Approve/publish a CIE-I notice targeted to FY MCA.
- 7. Switch to an FY MCA student account.
- 8. Show personalized dashboard with the notice and deadline.
- 9. Open notice and show original source + AI summary disclosure.
- 10. Open Events and register for a sample event.
- 11. Attempt a second registration to demonstrate duplicate protection.
- 12. Show notification and calendar.
- 13. Return to Admin and show audit log.

18. Definition of Done — Final Gate

- Application starts from a clean checkout using README instructions.
- Database migration works from empty database.
- Seed works with anonymized data.
- Student and admin authentication work.
- RBAC is server-enforced.
- Audience targeting works.
- Admin can publish an official notice.
- Eligible student sees it; ineligible student does not.
- PDF upload works.
- Text extraction works on text PDFs.
- OCR path works on a scanned fixture or fails gracefully.
- AI summary is validated and disclosed.
- AI cannot invent a published fact.
- Event registration is duplicate-safe and capacity-safe.
- Loading, empty and error states exist.
- Mobile layout works.
- Critical E2E tests pass.
- No real secrets or student PII are committed.
- Production build completes.
- Health endpoint works.
- Demo flow can be completed without manual database edits.

19. Agent Operating Rules

- Do not invent requirements that are not in the documents.
- Do not silently change database semantics.
- Do not expose secrets.
- Do not use real student PII in demo seed.

- Do not make AI the authority.
- Do not auto-publish OCR/AI results.
- Do not add unnecessary libraries.
- Do not replace backend rules with frontend checks.
- Do not mark a feature complete without testing its failure states.
- When a source document is ambiguous, preserve the original and flag for human review rather than guessing.

20. Team-of-Three Working Contract

Member	Primary ownership	Must review
Member 1	Product/UX + Frontend	Demo flow, UX, QA
Member 2	Backend + DB + Auth	API contracts, schema, security
Member 3	Frontend integration + QA + deployment	Responsive QA, E2E, demo/deployment

- All code goes through Git.
- Use short feature branches.
- Schema changes must be communicated to the team.
- Backend publishes stable endpoint contracts before integration where possible.
- Frontend may use mocked API data while backend is under construction.
- Every completed feature is tested from a real user's perspective.

21. Final Instruction to the Coding Agent

Build CampusFlow as a real, coherent product, not a collection of mock screens. Use the documents in /docs as the specification, the supplied academic documents as source fixtures, and this document as the final decision layer. Implement the MVP first, test each phase, preserve official source material, enforce permissions on the server, and keep AI/OCR assistive with human review. Do not ask the team to choose between the technology options listed in older documents; the choices in this handoff are final unless a build-breaking technical constraint is discovered.

When a technical constraint is discovered, explain the constraint, propose the smallest compatible change, and continue without changing product behavior.

22. Final Product Principle

CampusFlow should not try to become every college system. Its core job is to become the student's trusted layer for information, priority and action.